

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

Dezvoltarea unei platforme web
de management al proiectelor cu
funcționalități asistate de
inteligența artificială

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

Abstract

This thesis presents the design and implementation of *SDLC Hub*, a web platform for software project management. The application addresses a practical problem encountered in software teams: the need to work with different methodologies, preserve decision traceability, and connect development activity from GitHub with planning and project monitoring.

SDLC Hub supports Scrum, Kanban, and Scrumban. The selected methodology is not treated as a simple label: depending on the project mode, the interface, permissions, backlog, sprints, estimation fields, and reports are adapted dynamically. The application includes authentication, projects, roles, permissions, tasks, subtasks, comments, board, backlog, sprints, calendar, notifications, internal documentation, reporting, GitHub integration, real-time collaboration, and global administration.

An important contribution is the integration of AI-assisted features into concrete workflows: methodology recommendation, requirements refinement, story point estimation, risk analysis, workload balancing, and release notes generation. The thesis presents the motivation behind the application, related project management tools, the main requirements, the graphical interface, the architecture, the data model, the implementation details, and the validation of the implemented features.

Cuprins

1	Introducere	6
1.1	Studiu contextual	6
1.2	Motivație	7
1.3	Descriere	7
2	Aplicații similare	8
2.1	Jira Software	8
2.2	Trello	9
2.3	GitHub Projects	10
2.4	Analiză critică	11
3	Descrierea aplicației SDLC Hub	12
3.1	Cerințele aplicației	12
3.1.1	Diagrama cazurilor de utilizare	12
3.1.2	Detalii ale cazurilor de utilizare	13
3.2	Interfața grafică a aplicației (GUI)	16
3.2.1	Introducere	16
3.2.2	Lista paginilor	17
3.2.3	Elemente ale paginilor și analiză	17
3.2.4	Fluxul paginilor și navigarea	17
3.2.5	Capturi de ecran	18
3.2.6	Testare	24
3.3	Arhitectura aplicației	24
3.3.1	Frontend	24
3.3.2	Backend	25
3.3.3	Baza de date	25
3.3.4	Tehnologii folosite	26
4	Concluzii și direcții viitoare	27
4.1	Concluzii	27
4.2	Direcții viitoare	28
	Bibliografie	29

Lista figurilor

2.1	Interfață Jira Software, conform paginii oficiale Atlassian [4]	8
2.2	Pagina oficială Trello Guide [5]	9
2.3	Documentația oficială GitHub Projects [6]	10
3.1	Diagrama cazurilor de utilizare principale	13
3.2	Diagrama de secvență pentru autentificare	13
3.3	Diagrama de secvență pentru înregistrare	14
3.4	Diagrama de secvență pentru crearea proiectului	14
3.5	Diagrama de secvență pentru editarea task-ului	15
3.6	Diagrama de secvență pentru exportul raportului PDF	15
3.7	Diagrama de secvență pentru administrarea profilului	16
3.8	Diagrama de secvență pentru delogare	16
3.9	Fluxul de navigare în aplicație	18
3.10	Landing Page SDLC Hub	18
3.11	Login Page	19
3.12	Register Page	19
3.13	Project Wizard	20
3.14	Dashboard Page	20
3.15	Board Page	21
3.16	Tasks Page	21
3.17	Reports Page	22
3.18	Workload Page	22
3.19	Admin Console	23
3.20	Arhitectura sistemului SDLC Hub	24
3.21	Modelul conceptual de date	26

Capitolul 1

Introducere

1.1 Studiu contextual

Managementul proiectelor software reprezintă o componentă importantă a ciclului de viață al dezvoltării software. O echipă care construiește un produs informatic trebuie să poată transforma cerințele în activități concrete, să urmărească progresul, să colaboreze, să controleze schimbările și să păstreze o evidență a deciziilor luate. În acest context, instrumentul de management folosit de echipă nu este doar o listă de task-uri, ci un spațiu în care planificarea, execuția, comunicarea și raportarea se întâlnesc.

În practică, echipele software nu lucrează toate după aceeași metodologie. Scrum organizează munca în sprinturi, roluri, evenimente și artefacte, punând accent pe planificare iterativă și inspecție regulată [1]. Kanban pune accent pe vizualizarea fluxului de lucru, limitarea lucrului în desfășurare și îmbunătățirea continuă a procesului [2]. Între aceste două direcții apare Scrumban, o abordare hibridă care permite folosirea unor elemente de planificare din Scrum împreună cu o execuție mai flexibilă, apropiată de Kanban.

Pe lângă metodologia aleasă, echipele au nevoie de trasabilitate. Un task poate fi creat dintr-o cerință, poate fi discutat în comentarii, poate fi estimat, mutat pe board, implementat într-un branch GitHub și finalizat printr-un pull request. Dacă aceste informații se află în instrumente separate, contextul se pierde ușor. De aceea, o platformă modernă trebuie să lege activitatea de management de activitatea tehnică și să permită urmărirea întregului traseu al muncii.

În ultimii ani, inteligența artificială a început să fie folosită tot mai des în activități de dezvoltare software, inclusiv pentru analiză, sumarizare, rafinare de cerințe și estimări [3]. Totuși, aceste funcții sunt utile doar dacă sunt integrate într-un flux real. O sugestie AI izolată, plasată într-o pagină separată, nu ajută suficient echipa. În schimb, dacă AI-ul apare în momentul în care utilizatorul creează un proiect, rafinează o cerință, estimează un task sau analizează un risc, valoarea lui devine practică.

1.2 Motivație

Motivația acestei lucrări pornește de la observația că multe aplicații de management al proiectelor sunt fie foarte complexe, fie prea simple pentru un flux software complet. O aplicație complexă poate oferi multe opțiuni, dar configurarea ei poate fi dificilă pentru o echipă mică sau pentru un proiect universitar. O aplicație simplă este ușor de folosit, dar nu oferă întotdeauna roluri, permisiuni, audit, rapoarte, documentație internă sau integrare cu activitatea din cod.

O altă motivație este flexibilitatea metodologică. Într-un proiect real, echipa poate începe cu Scrum, poate observa că are nevoie de un flux continuu și poate trece spre Kanban sau Scrumban. Dacă aplicația tratează metodologia doar ca etichetă, schimbarea nu are efect real. Dacă metodologia controlează paginile, câmpurile, regulile și rapoartele, instrumentul devine mai apropiat de modul real de lucru al echipei.

Lucrarea urmărește realizarea unei platforme web care să combine managementul proiectelor software, adaptarea la metodologii agile și funcționalități asistate de inteligența artificială. Scopul nu este reproducerea completă a unui produs enterprise matur, ci construirea unei aplicații coerente, demonstrabile, în care principalele fluxuri de lucru pot fi parcurse de la început până la final.

1.3 Descriere

Soluția propusă este *SDLC Hub*, o platformă web pentru managementul proiectelor software. Utilizatorul poate crea cont, se poate autentifica, poate crea un proiect sau poate intra într-un proiect existent. La crearea proiectului, aplicația oferă un wizard care ajută la definirea identității proiectului, alegerea metodologiei, configurarea rolurilor și pregătirea integrării DevOps.

Aplicația suportă Scrum, Kanban și Scrumban. Metodologia selectată influențează interfața și regulile sistemului. Pentru Scrum și Scrumban sunt disponibile backlog-ul și sprinturile, iar pentru Kanban accentul cade pe board și pe flux continuu. Schimbarea metodologiei este controlată prin preview, blocaje, avertismente și audit log.

SDLC Hub include module pentru task-uri, subtasks, comentarii, board, backlog, sprinturi, roluri, permisiuni, audit, calendar, notificări, documentație internă, rapoarte, workload, integrare GitHub, suport, administrare globală și funcționalități AI. Backend-ul este construit cu FastAPI, SQLAlchemy, Alembic și PostgreSQL, iar frontend-ul folosește Next.js, React, TypeScript și Tailwind CSS. Comunicarea dintre frontend și backend se face prin API REST, iar actualizările live sunt realizate prin WebSocket.

Capitolul 2

Aplicații similare

2.1 Jira Software

Jira Software este una dintre cele mai cunoscute aplicații pentru managementul proiectelor și este dezvoltată de Atlassian. Pagina oficială de funcționalități prezintă Jira ca instrument pentru planificare, urmărirea progresului, colaborare, automatizare de workflow și raportare [4]. Jira oferă mai multe tipuri de vizualizări, precum board, listă, calendar și timeline, iar pentru echipele software include funcții utile pentru sprinturi, rapoarte, dependențe, câmpuri configurabile și integrări.

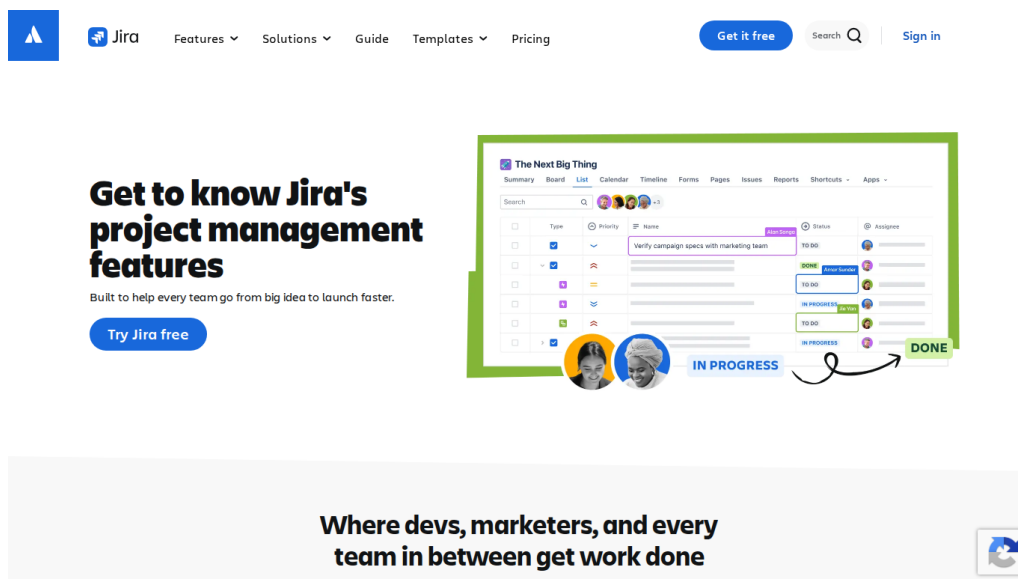


Figura 2.1: Interfață Jira Software, conform paginii oficiale Atlassian [4]

Avantaje. Principalul avantaj al Jira este nivelul ridicat de configurare. Platforma permite definirea de workflow-uri, câmpuri, permisiuni, rapoarte și integrări,

fiind potrivită pentru organizații care au procese complexe. Jira este puternică și prin ecosistemul Atlassian, deoarece se poate conecta cu alte instrumente precum Confluence, Bitbucket sau aplicații din Atlassian Marketplace.

Dezavantaje. Complexitatea poate deveni o problemă pentru echipe mici sau pentru proiecte care au nevoie de configurare rapidă. Multe funcții sunt utile, dar pot încălca experiența utilizatorului. Din perspectiva SDLC Hub, Jira este un reper important pentru board, sprint management și raportare, însă aplicația propusă urmărește o experiență mai directă, cu tranziție explicită între Scrum, Kanban și Scrumban.

2.2 Trello

Trello este o aplicație orientată spre organizare vizuală prin board-uri, liste și carduri. Ghidul oficial Trello descrie procesul de început ca fiind simplu: utilizatorul se înregistrează, creează un board și poate începe să organizeze munca echipei [5]. Trello include funcții precum template-uri, automatizări, Power-Ups, integrări, planificare și mai multe moduri de folosire pentru echipe diferite.

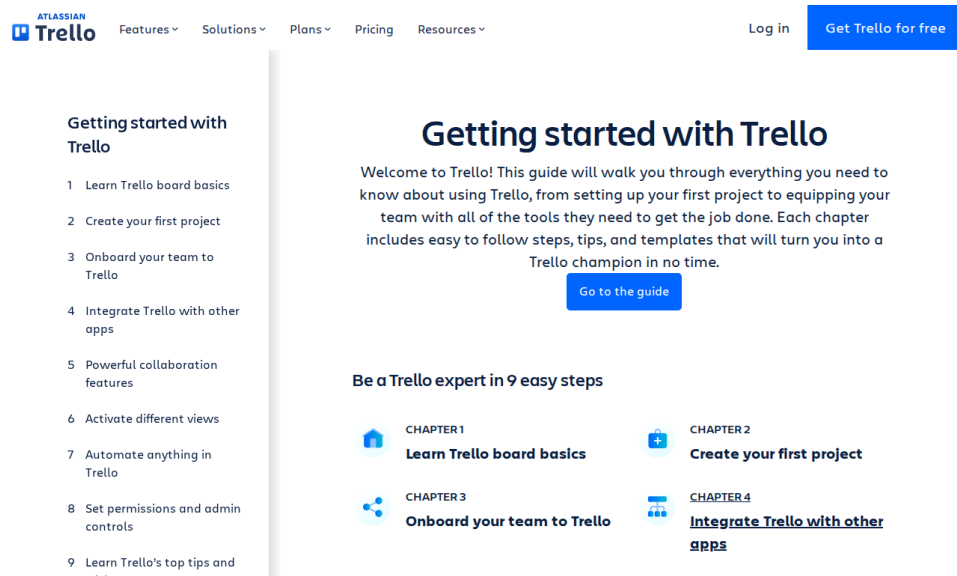


Figura 2.2: Pagina oficială Trello Guide [5]

Avantaje. Trello este ușor de înțeles și de folosit. Metafora cardurilor mutate între liste este naturală pentru utilizatori și se potrivește foarte bine cu un flux Kanban simplu. Platforma este potrivită pentru echipe mici, proiecte personale și activități unde vizibilitatea rapidă este mai importantă decât configurarea avansată.

Dezavantaje. Trello nu este construit special pentru toate nevoile unui proces software complet. Auditul detaliat, permisiunile granulare, sprint management-ul,

rapoartele agile și integrarea profundă cu task-uri tehnice necesită extensii sau soluții suplimentare. SDLC Hub păstrează claritatea vizuală a unui board, dar adaugă roluri, metodologii, audit, rapoarte, documentație și integrare DevOps.

2.3 GitHub Projects

GitHub Projects este integrat în ecosistemul GitHub și permite planificarea muncii direct lângă issues și pull request-uri. Documentația oficială descrie Projects ca un instrument flexibil pentru planificare și urmărirea muncii, bazat pe tabele, board-uri și roadmap-uri [6]. Proiectele pot avea vizualizări personalizate, filtre, câmpuri custom, grafice, template-uri și automatizări.

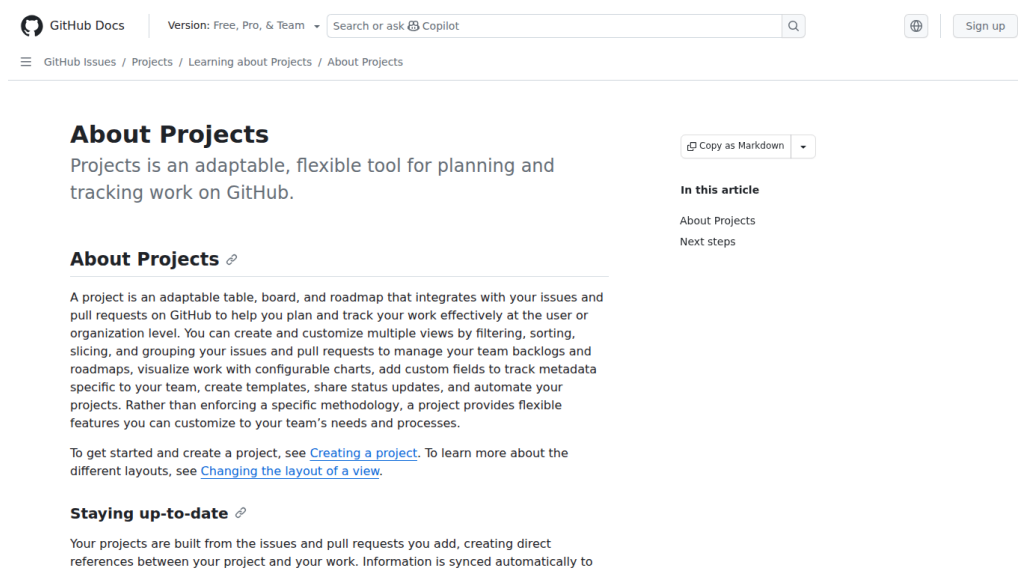


Figura 2.3: Documentația oficială GitHub Projects [6]

Avantaje. Cel mai mare avantaj este apropierea de cod. Issues, pull request-urile și proiectele sunt conectate, iar actualizările pot fi sincronizate între ele. Pentru echipe tehnice, această apropiere reduce distanța dintre planificare și implementare. GitHub Projects este util și prin flexibilitatea vizualizărilor: tabel, board și roadmap.

Dezavantaje. GitHub Projects este mai puțin accesibil pentru utilizatori non-tehnici și nu tratează managementul proiectului ca spațiu separat de repository. Rapoartele, rolurile de proiect, tranzițiile metodologice și funcțiile AI nu sunt elemente centrale ale aplicației. SDLC Hub preia ideea de trasabilitate între task și cod prin webhook-uri GitHub, dar păstrează managementul proiectului într-o platformă dedicată.

2.4 Analiză critică

Jira, Trello și GitHub Projects acoperă direcții importante. Jira este foarte configurabilă, Trello este simplă și vizuală, iar GitHub Projects este aproape de cod. Totuși, fiecare aplicație are și limite. Jira poate deveni dificilă pentru echipe mici, Trello nu acoperă suficient un proces software complet, iar GitHub Projects este strâns legat de repository și de utilizatori tehnici.

SDLC Hub este construit pornind de la aceste observații. Aplicația propusă preia ideea de board vizual, dar o leagă de task-uri, sprinturi, roluri, permisiuni și audit. Preia ideea de apropiere de cod, dar prin webhooks și evenimente GitHub, nu prin mutarea întregului management în repository. Preia ideea de raportare, dar o adaptează metodologiei active. În plus, metodologia nu este doar un câmp informativ, ci influențează paginile și regulile aplicației. Studiile comparative asupra Scrum și Kanban arată că modul de lucru poate influența performanța proiectelor software [7], motiv pentru care metodologia trebuie tratată ca parte activă a sistemului.

Capitolul 3

Descrierea aplicației SDLC Hub

3.1 Cerințele aplicației

Principalele cerințe ale aplicației sunt ca un utilizator să poată crea un cont, să se autentifice, să creeze un proiect, să aleagă o metodologie, să invite membri, să gestioneze task-uri și să urmărească progresul proiectului. Aplicația trebuie să ofere un flux complet pentru echipe software, nu doar o pagină de board. Din acest motiv, SDLC Hub include și roluri, permisiuni, sprinturi, calendar, documentație, notificări, rapoarte, workload, integrare GitHub și funcții AI.

O cerință importantă este adaptarea la metodologia activă. Pentru Scrum, aplicația trebuie să permită backlog, sprint planning, pornire sprint, închidere sprint și rapoarte de sprint. Pentru Kanban, aplicația trebuie să susțină un board continuu și să ascundă operațiile specifice sprinturilor. Pentru Scrumban, sistemul trebuie să permită combinarea planificării cu fluxul vizual. Schimbarea metodologiei trebuie să fie controlată prin preview, astfel încât utilizatorul să vadă impactul înainte de aplicare.

Din punct de vedere al securității, sistemul trebuie să verifice rolurile și permisiunile în backend. Interfața poate ascunde acțiunile nepermise, dar backend-ul trebuie să refuze cererile neautorizate. Pentru acțiunile importante, aplicația trebuie să păstreze audit log, astfel încât să fie clar cine a făcut modificarea și când.

3.1.1 Diagrama cazurilor de utilizare

Figura 3.1 prezintă actorii principali și interacțiunile lor cu sistemul. Utilizatorul obișnuit lucrează cu proiecte, task-uri, board, calendar și rapoarte. Project Admin controlează metodologia, rolurile, permisiunile și integrarea GitHub. Global Admin are acces la consola globală, iar sistemele externe sunt reprezentate de GitHub și serviciul AI.

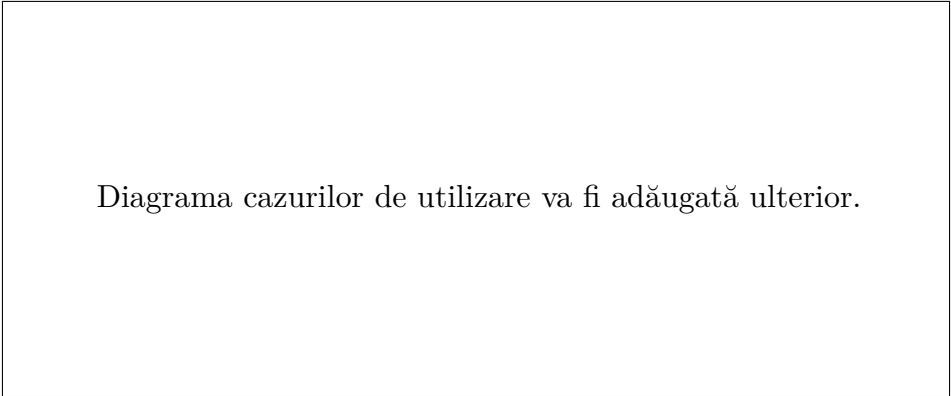


Diagrama cazurilor de utilizare va fi adăugată ulterior.

Figura 3.1: Diagrama cazurilor de utilizare principale

3.1.2 Detalii ale cazurilor de utilizare

În continuare sunt prezentate cele mai importante cazuri de utilizare ale aplicației. Formatul urmărește același tip de descriere folosit în modelul de referință: actor, descriere, precondiții, postcondiții și diagramă de secvență.

Login. Actorul este utilizatorul. Utilizatorul se poate autentifica în aplicația SDLC Hub folosind e-mailul și parola contului său. Precondiția este ca utilizatorul să aibă un cont creat și valid, iar postcondiția este autentificarea cu succes și accesul la proiectele din care face parte.

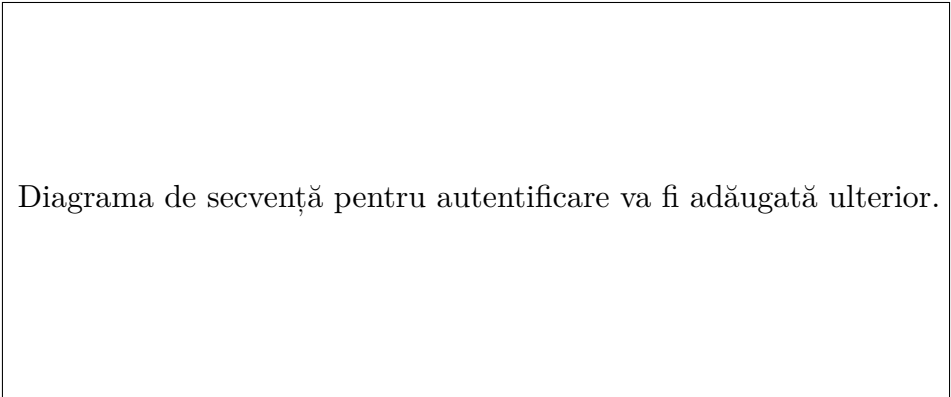


Diagrama de secvență pentru autentificare va fi adăugată ulterior.

Figura 3.2: Diagrama de secvență pentru autentificare

Register. Actorul este utilizatorul nou. Acesta poate crea un cont în SDLC Hub completând formularul de înregistrare. Precondiția este accesul la pagina de înregistrare, iar postcondiția este crearea contului și trimiterea e-mailului de verificare.

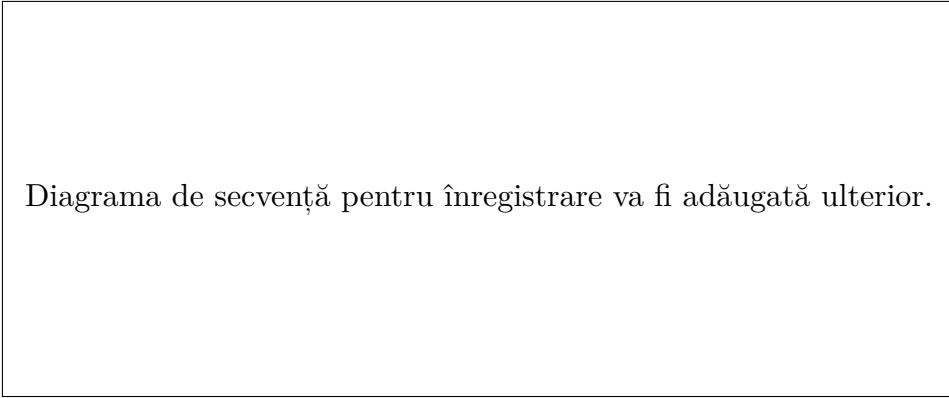


Diagrama de secvență pentru înregistrare va fi adăugată ulterior.

Figura 3.3: Diagrama de secvență pentru înregistrare

Create Project. Actorul este utilizatorul autentificat. Utilizatorul poate crea un proiect nou prin Project Wizard, completând datele de bază, alegând metodologia, configurând rolurile și invitând membri. Precondiția este ca utilizatorul să fie autentificat, iar postcondiția este crearea proiectului și atașarea utilizatorului ca Project Admin.

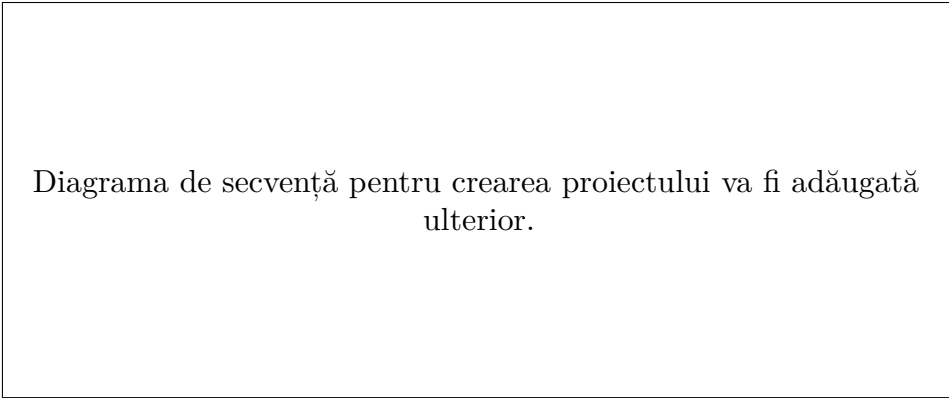


Diagrama de secvență pentru crearea proiectului va fi adăugată ulterior.

Figura 3.4: Diagrama de secvență pentru crearea proiectului

Edit Task. Actorul este membrul proiectului. Utilizatorul poate edita un task existent, modificând titlul, descrierea, statusul, prioritatea, estimarea, persoana asignată sau sprintul. Precondiția este existența unui task și a permisiunii de editare, iar postcondiția este actualizarea task-ului și înregistrarea modificării în audit log.

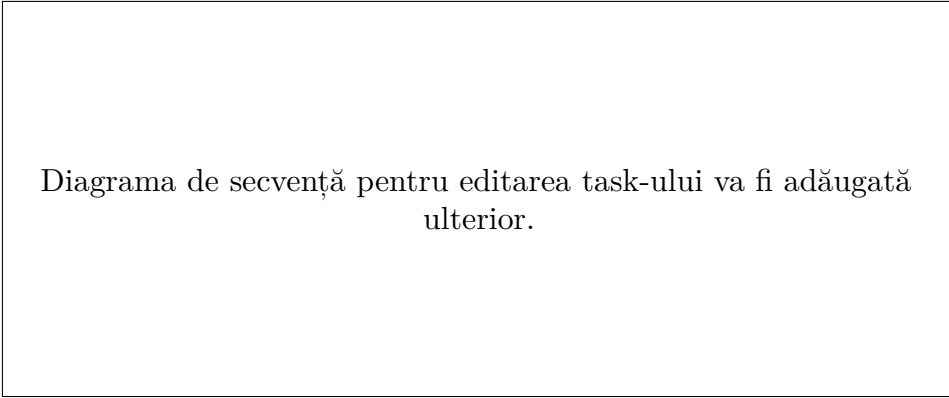


Diagrama de secvență pentru editarea task-ului va fi adăugată ulterior.

Figura 3.5: Diagrama de secvență pentru editarea task-ului

Export PDF Report. Actorul este utilizatorul cu drept de vizualizare a rapoartelor. Utilizatorul poate genera un raport PDF al stării proiectului, folosind datele din task-uri, sprinturi și audit log. Precondiția este existența unui proiect cu date raportabile, iar postcondiția este generarea fișierului PDF pentru distribuire sau arhivare.

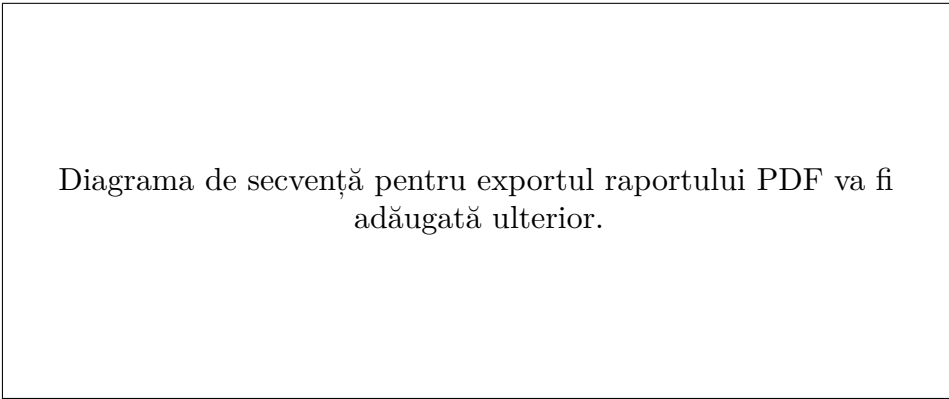


Diagrama de secvență pentru exportul raportului PDF va fi adăugată ulterior.

Figura 3.6: Diagrama de secvență pentru exportul raportului PDF

Manage Profile. Actorul este utilizatorul autentificat. Utilizatorul poate vizualiza și modifica datele contului, parola, avatarul, preferințele de notificare și sesiunile active. Precondiția este autentificarea, iar postcondiția este actualizarea informațiilor de profil în aplicație.

Diagrama de secvență pentru administrarea profilului va fi adăugată ulterior.

Figura 3.7: Diagrama de secvență pentru administrarea profilului

Logout. Actorul este utilizatorul autentificat. Acesta apasă butonul de delogare, iar frontend-ul șterge token-ul local și îl redirecționează spre zona publică. Precondiția este existența unei sesiuni active, iar postcondiția este imposibilitatea accesării paginilor private până la o nouă autentificare.

Diagrama de secvență pentru delogare va fi adăugată ulterior.

Figura 3.8: Diagrama de secvență pentru delogare

3.2 Interfața grafică a aplicației (GUI)

3.2.1 Introducere

Interfața grafică a aplicației urmărește să fie clară, modernă și potrivită pentru utilizare zilnică. Utilizatorul trebuie să poată vedea rapid proiectul curent, metodologia activă, task-urile, rapoartele și notificările. După autentificare, zona principală este organizată în jurul unui dashboard cu sidebar, iar modulele disponibile se adaptează la metodologia proiectului și la permisiunile utilizatorului.

Scopul interfeței este să ghideze utilizatorul fără să îl oblige să parcurgă multe ecrane pentru acțiuni obișnuite. De exemplu, board-ul poate fi accesat direct din dashboard, task-urile pot fi deschise din listă sau din board, iar rapoartele pot fi

exportate în PDF. Pentru proiectele Kanban, elementele specifice sprinturilor nu sunt tratate ca zonă centrală, iar pentru proiectele Scrum și Scrumban backlog-ul și sprinturile sunt disponibile.

3.2.2 Lista paginilor

Aplicația conține pagini publice și pagini private. Paginile publice sunt Landing Page, Login Page, Register Page, Verify Email Page, Forgot Password Page și Reset Password Page. Acestea permit prezentarea aplicației, crearea contului, autentificarea și recuperarea accesului.

După autentificare, utilizatorul poate ajunge în Onboarding sau în Project Wizard. Spațiul principal de lucru include Dashboard, Board, Backlog, Tasks, Task Detail, Activity, Calendar, Documentation, Team, Reports, Workload, DevOps, Pull Requests, Notifications, Support, Settings și Account. Pentru administratorul global există și pagina Admin Console, care afișează utilizatori, proiecte, suport, erori HTTP și consum AI.

3.2.3 Elemente ale paginilor și analiză

Landing Page prezintă aplicația și oferă acces rapid către Login și Register. Login Page conține câmpuri pentru e-mail și parolă, iar Register Page permite crearea unui cont nou. Project Wizard conține pași pentru identitatea proiectului, interviul AI, alegerea metodologiei, configurarea echipei și pregătirea integrării DevOps.

Dashboard-ul este pagina principală după intrarea în proiect. El afișează indicatori precum velocity, completion, active work, team health, task-uri overdue, due soon, critical open și backlog. Board-ul afișează task-urile pe coloane și permite mutarea lor între statusuri. Tasks Page oferă o vedere tabelară sau listă asupra task-urilor, cu filtre și acțiuni de căutare. Reports Page prezintă date agregate despre progres, story points, lead time, bottlenecks și WIP. Workload Page ajută la identificarea membrilor supraîncărcați și a task-urilor neasignate.

Documentation Page păstrează pagini interne ale proiectului, iar unele documente pot fi generate din task-uri finalizate. Calendar Page afișează evenimente, disponibilitate și termene. Settings Page permite administrarea proiectului, a metodologiei, a rolurilor și a integrărilor. Account Page conține datele utilizatorului, preferințele, sesiuni active și security log. Admin Console este folosită pentru supravegherea globală a platformei.

3.2.4 Fluxul paginilor și navigarea

Figura 3.9 prezintă fluxul principal de navigare. Utilizatorul pornește din zona publică, trece prin autentificare și onboarding, iar după selectarea proiectului ajunge în dashboard. De acolo poate accesa modulele principale ale proiectului.

Diagrama fluxului dintre paginile aplicației va fi adăugată ulterior.

Figura 3.9: Fluxul de navigare în aplicație

Diagrama de mai sus arată traseul principal al utilizatorului. Utilizatorul pornește din zona publică, se autentifică sau se înregistrează, ajunge în onboarding sau în Project Wizard, iar apoi intră în dashboard. Din dashboard poate naviga spre modulele principale ale proiectului.

3.2.5 Capturi de ecran

În această secțiune sunt prezentate capturi din aplicația SDLC Hub. Capturile au fost realizate din aplicația pornită local și arată atât paginile publice, cât și paginile autentificate.

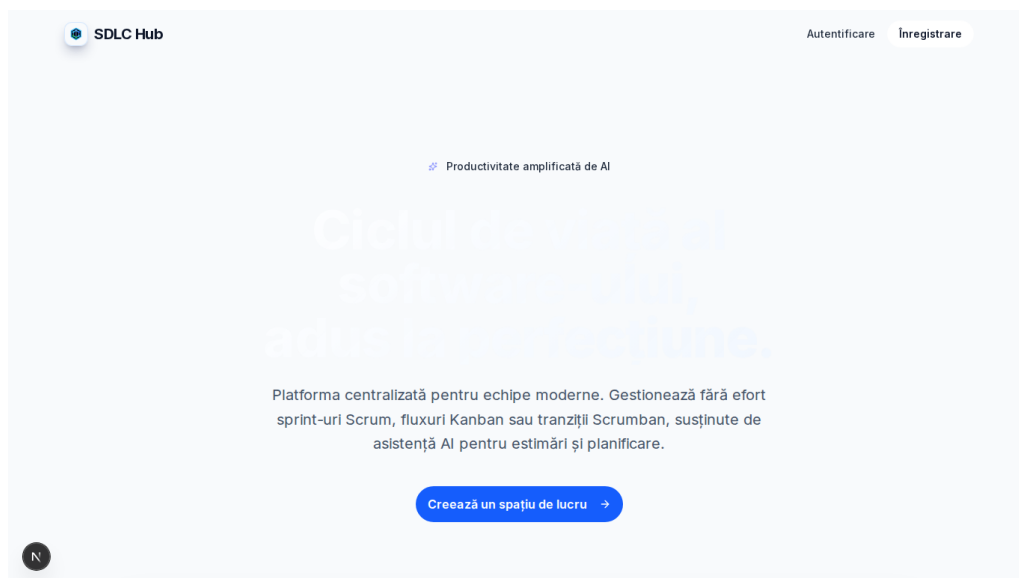
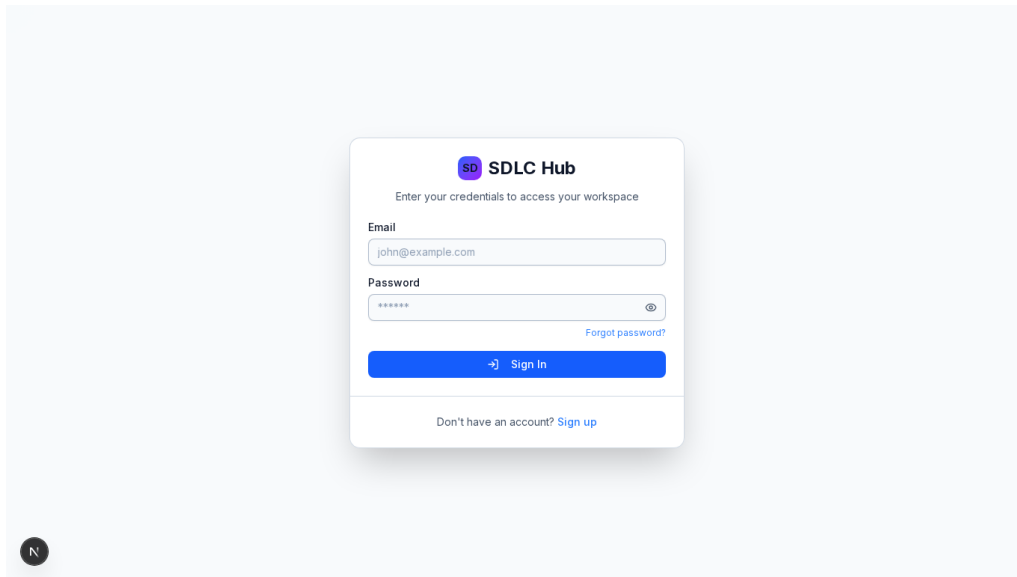


Figura 3.10: Landing Page SDLC Hub



The login page for SDLC Hub features a central white card on a light blue background. At the top of the card is the SDLC Hub logo, consisting of a purple circle with 'SD' and the text 'SDLC Hub'. Below the logo is the instruction 'Enter your credentials to access your workspace'. The form includes an 'Email' field with the placeholder 'john@example.com' and a 'Password' field with masked characters '*****'. A 'Forgot password?' link is positioned to the right of the password field. A blue 'Sign In' button is centered below the fields. At the bottom of the card, a link for users without an account is provided. A small circular icon with the letter 'N' is located in the bottom-left corner of the page.

SDLC Hub

Enter your credentials to access your workspace

Email

john@example.com

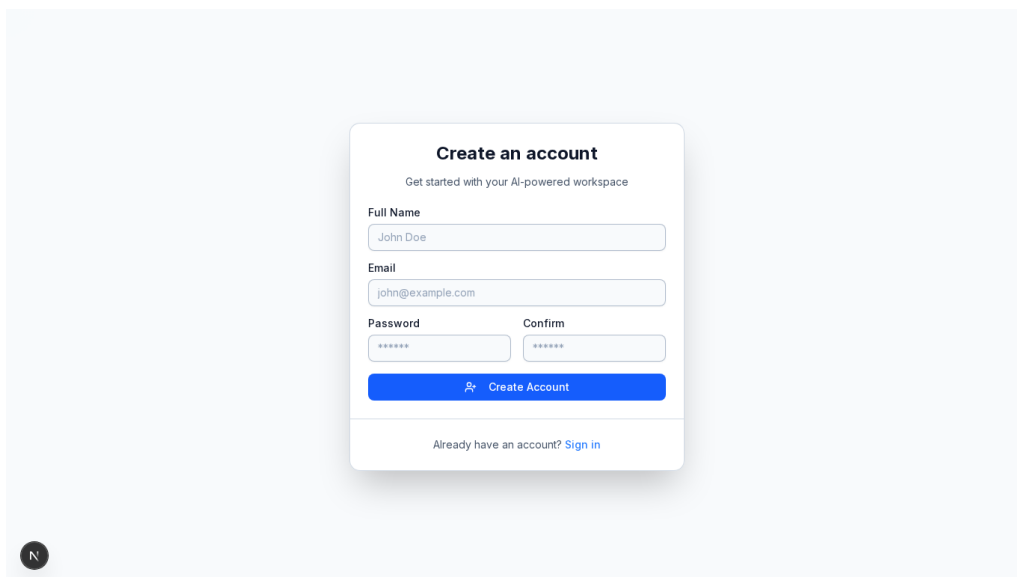
Password

[Forgot password?](#)

[Sign In](#)

Don't have an account? [Sign up](#)

Figura 3.11: Login Page



The registration page for SDLC Hub features a central white card on a light blue background. At the top of the card is the heading 'Create an account' followed by the subtext 'Get started with your AI-powered workspace'. The form includes a 'Full Name' field with the placeholder 'John Doe', an 'Email' field with the placeholder 'john@example.com', a 'Password' field with masked characters '*****', and a 'Confirm' field with masked characters '*****'. A blue 'Create Account' button is centered below the fields. At the bottom of the card, a link for existing users is provided. A small circular icon with the letter 'N' is located in the bottom-left corner of the page.

Create an account

Get started with your AI-powered workspace

Full Name

John Doe

Email

john@example.com

Password

Confirm

[Create Account](#)

Already have an account? [Sign in](#)

Figura 3.12: Register Page

Identity

AI Interview

Decision

Team & Roles

DevOps Setup

x

Project Basics

Establish your project identity.

Project Logo

Click to Upload

Project Name

e.g. SuperApp Rewrite

Key

APP

Short Description

Briefly describe the goal...

Next Step

→

Figura 3.13: Project Wizard

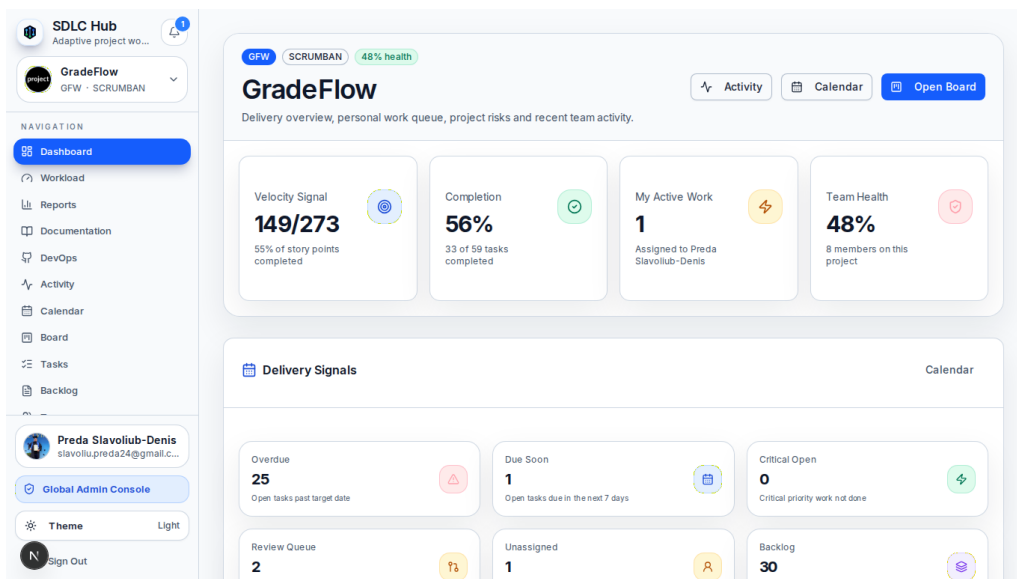


Figura 3.14: Dashboard Page

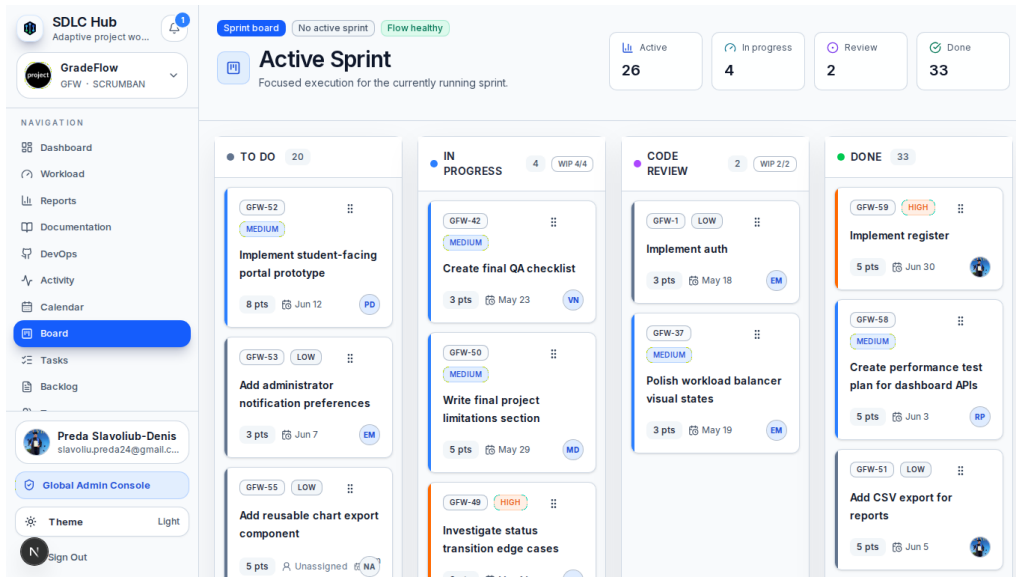


Figura 3.15: Board Page

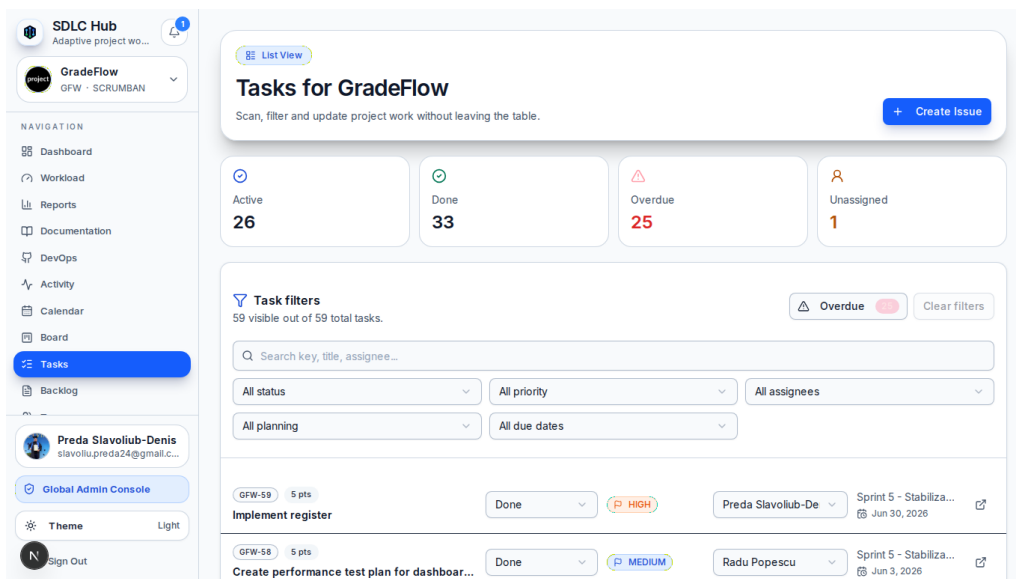


Figura 3.16: Tasks Page

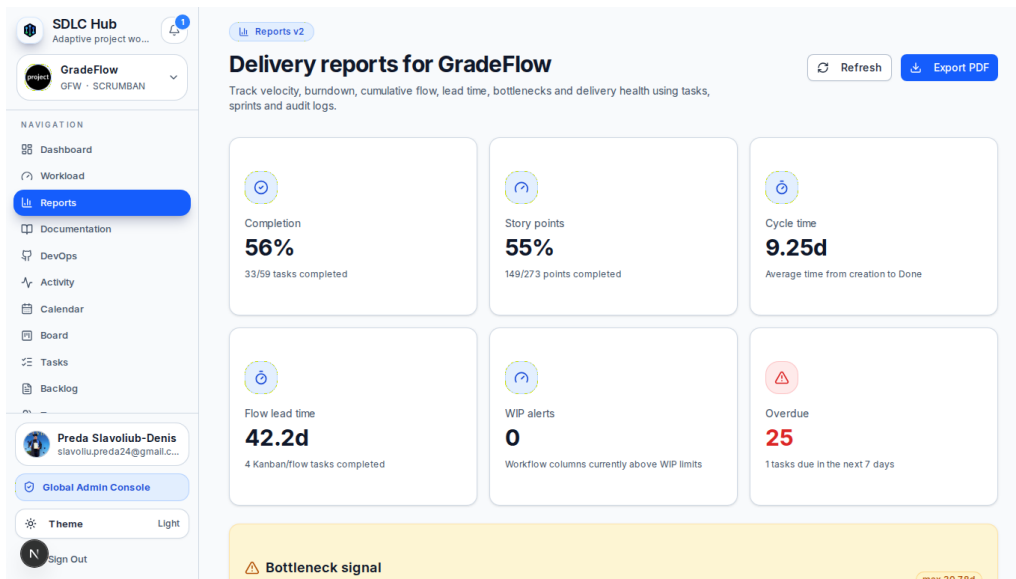


Figura 3.17: Reports Page

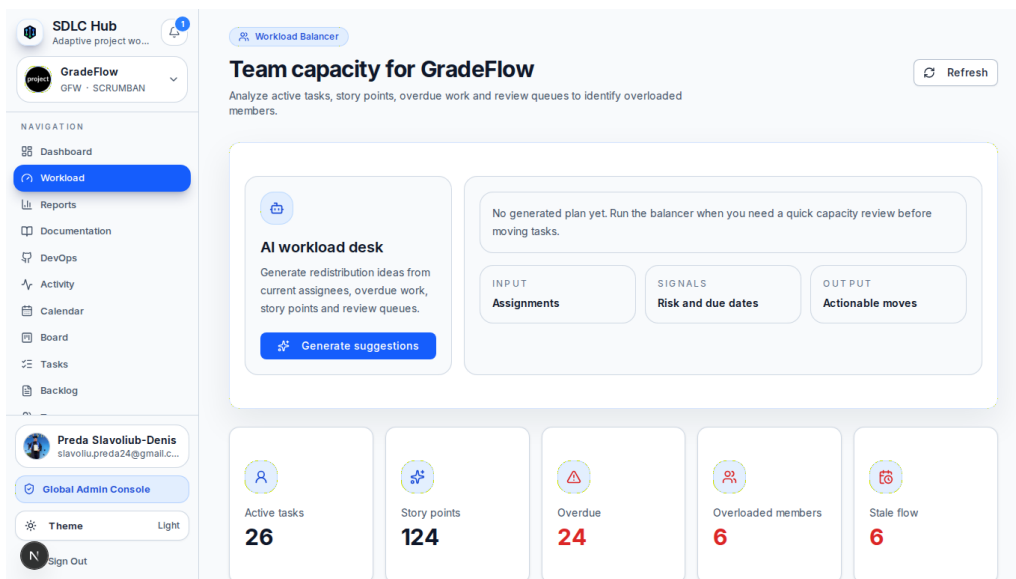


Figura 3.18: Workload Page

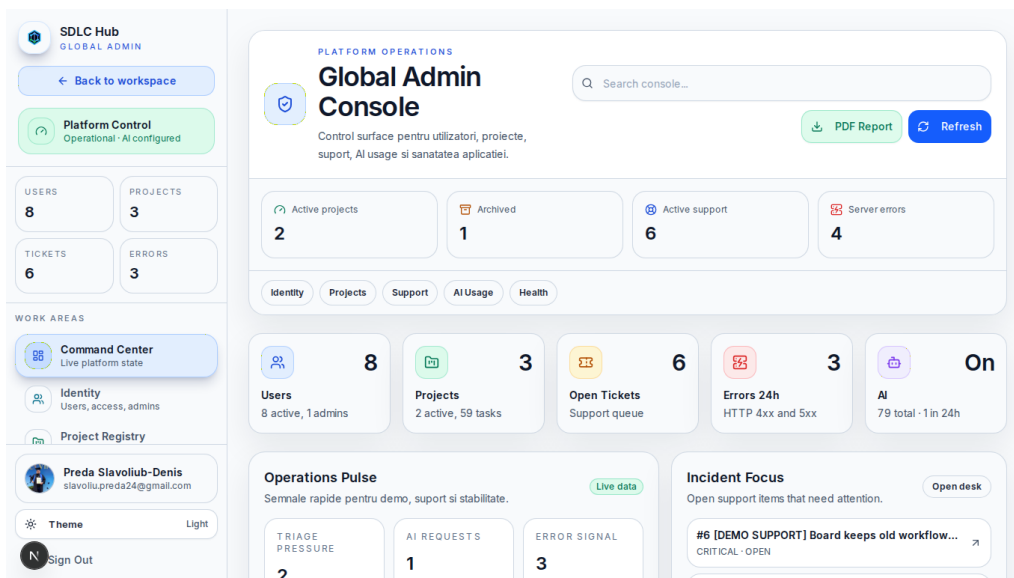


Figura 3.19: Admin Console

3.2.6 Testare

Pentru a verifica funcționarea aplicației au fost realizate teste manuale și verificări automate headless. Au fost parcurse paginile publice, paginile autentificate, fluxurile de autentificare, proiecte, task-uri, sprinturi, board, rapoarte, workload, calendar, notificări, documentație, integrare GitHub și administrare globală.

Testarea a urmărit atât scenarii pozitive, cât și scenarii negative. La autentificare au fost verificate date corecte și date greșite. La proiecte au fost verificate încărcarea proiectelor, schimbarea metodologiei și blocarea tranziției atunci când există un sprint activ. La task-uri au fost verificate creare, editare, mutare, subtasks, comentarii și audit. Pentru GitHub au fost verificate statusul integrării, evenimentele și lista de pull request-uri. Pentru interfață au fost verificate variante desktop și mobile, light mode și dark mode.

3.3 Arhitectura aplicației

Aplicația folosește o arhitectură pe trei niveluri: frontend, backend și bază de date. Acest model separă interfața utilizatorului, logica de business și persistența datelor, ceea ce ușurează dezvoltarea și testarea sistemului [8].

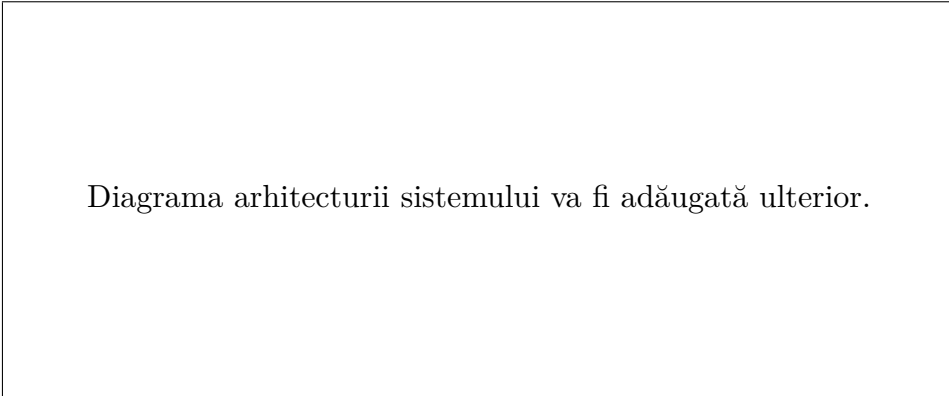


Diagrama arhitecturii sistemului va fi adăugată ulterior.

Figura 3.20: Arhitectura sistemului SDLC Hub

Frontend-ul comunică cu backend-ul prin API REST, iar backend-ul comunică cu PostgreSQL prin SQLAlchemy. Actualizările live sunt transmise prin WebSocket. Integrarea GitHub se face prin webhook-uri verificate HMAC, iar e-mailurile sunt trimise prin SMTP. Funcțiile AI sunt centralizate într-un serviciu backend care poate folosi un provider extern sau un fallback local.

3.3.1 Frontend

Frontend-ul este construit cu Next.js, React, TypeScript și Tailwind CSS [9, 10, 11]. Structura este organizată pe rute, componente și servicii API. Paginile din

`frontend/app` formează ecranele aplicației, componentele reutilizabile sunt păstrate în `frontend/components`, iar comunicarea cu backend-ul este centralizată în `frontend/services`. Pentru interacțiuni vizuale sunt folosite biblioteci precum `dnd-kit` pentru drag and drop [12], `Recharts` pentru grafice [13] și `Tiptap` pentru editorul rich text [14].

O parte importantă a frontend-ului este adaptarea la contextul proiectului. Componentele nu afișează aceleași acțiuni pentru toate metodologiile. Pagina `Backlog` este ascunsă pentru Kanban, acțiunile de sprint sunt disponibile doar când metodologia le permite, iar câmpurile de estimare sunt tratate diferit în funcție de modul de lucru. `TypeScript` ajută la păstrarea contractelor de date între serviciile API și componente.

3.3.2 Backend

Backend-ul este construit cu `FastAPI` [15]. El expune endpoint-uri REST pentru autentificare, proiecte, task-uri, sprinturi, calendar, notificări, documentație, echipe, rapoarte, GitHub și admin global. `SQLAlchemy` este folosit pentru accesul la baza de date [16], iar `Alembic` gestionează migrațiile [17]. Autentificarea folosește `JWT` [18], iar verificarea permisiunilor se face la nivel de endpoint.

Backend-ul este responsabil pentru regulile de business. Dacă frontend-ul trimite o cerere pentru crearea unui sprint într-un proiect Kanban, backend-ul refuză operația. Dacă un utilizator încearcă să mute un task fără permisiune, backend-ul returnează eroare. Dacă un webhook GitHub este primit fără semnătură validă, evenimentul nu este procesat. Astfel, aplicația rămâne sigură chiar dacă interfața este ocolită.

3.3.3 Baza de date

Datele sunt persistate în `PostgreSQL` [19]. Modelul include utilizatori, proiecte, membri, roluri, echipe, task-uri, subtasks, comentarii, sprinturi, audit log-uri, evenimente calendaristice, disponibilitate, notificări, documentație, integrări GitHub, suport tickets, loguri HTTP și loguri de consum AI. Această structură permite păstrarea separată a datelor operaționale, a istoricului și a integrărilor externe.

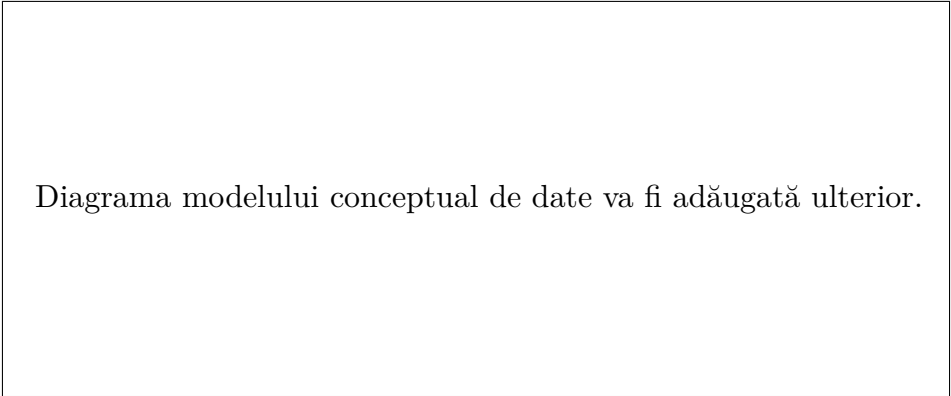


Diagrama modelului conceptual de date va fi adăugată ulterior.

Figura 3.21: Modelul conceptual de date

Relația dintre **User**, **Project** și **ProjectMember** permite ca același utilizator să participe la mai multe proiecte, cu roluri diferite. Astfel, un utilizator poate fi **Project Admin** într-un proiect și membru obișnuit în altul. Această flexibilitate este necesară într-o platformă multi-proiect, deoarece drepturile nu aparțin utilizatorului în mod global, ci depind de contextul proiectului.

Entitatea **Task** este legată de mai multe zone ale sistemului. Ea poate aparține unui sprint, poate avea o echipă de livrare, poate genera audit log, poate fi menționată în documentație și poate primi evenimente GitHub. Această poziție centrală justifică faptul că multe module ale aplicației, precum board, reports, workload, documentation și DevOps, pornesc de la starea task-urilor.

3.3.4 Tehnologii folosite

Pentru backend au fost folosite Python, FastAPI, Uvicorn, SQLAlchemy, Alembic, PostgreSQL, JWT și fpdf2. Pentru frontend au fost folosite Next.js, React, TypeScript, Tailwind CSS, componente shadcn/Radix, TanStack Query [20], dnd-kit, Recharts, Tiptap și Lucide React. Integrarea externă se bazează pe GitHub Webhooks [21], WebSocket pentru evenimente live [22], SMTP pentru e-mail și ngrok pentru testarea locală a webhook-urilor.

Alegerea acestor tehnologii a urmărit echilibrul dintre productivitate și robustețe. FastAPI permite dezvoltarea rapidă a unui API clar, dar oferă și performanță bună pentru un backend asincron. SQLAlchemy și Alembic oferă control asupra modelului de date și asupra evoluției schemei. PostgreSQL este potrivit pentru relații complexe și pentru integritatea datelor.

Pe frontend, Next.js și React au permis organizarea aplicației pe rute și componente. Tailwind CSS a ajutat la construirea unei interfețe consistente, iar bibliotecile specializate au fost folosite acolo unde era nevoie de comportamente complexe: drag and drop pentru board, grafice pentru rapoarte și editor rich text pentru descrieri și documentație.

Capitolul 4

Concluzii și direcții viitoare

4.1 Concluzii

Scopul lucrării a fost realizarea unei platforme web de management al proiectelor software, cu suport pentru metodologii diferite și funcționalități asistate de inteligența artificială. Obiectivul a fost atins prin dezvoltarea aplicației SDLC Hub, care oferă fluxuri complete pentru autentificare, proiecte, task-uri, sprinturi, board, roluri, permisiuni, audit, calendar, notificări, documentație, rapoarte, GitHub, realtime și admin global.

Cea mai importantă contribuție este tratarea metodologiei ca parte activă a sistemului. Aplicația nu doar afișează numele metodologiei, ci modifică paginile, regulile, câmpurile și rapoartele în funcție de Scrum, Kanban sau Scrumban. În plus, tranziția între metodologii este făcută controlat, cu preview și audit.

O altă contribuție este integrarea AI în fluxuri practice. AI-ul nu este prezentat ca element decorativ, ci este folosit pentru recomandări, rafinarea cerințelor, estimări, riscuri, workload și release notes. Rezultatele sunt editabile de utilizator, ceea ce păstrează controlul uman asupra deciziilor.

Prin arhitectura aleasă, aplicația separă clar interfața, logica de business și persistența datelor. Această separare a permis integrarea mai multor module fără ca sistemul să devină dependent de o singură pagină sau de un singur flux. Task-urile sunt legate de sprinturi, board, comentarii, audit, documentație și GitHub, iar această legătură oferă trasabilitate pe întregul parcurs al muncii.

Validarea realizată arată că aplicația poate fi folosită pentru scenarii complete de proiect. Au fost testate autentificarea, proiectele, task-urile, sprinturile, schimbarea metodologiei, rolurile, calendarul, notificările, documentația, GitHub, rapoartele și administrarea globală. Rezultatul nu este doar o colecție de ecrane, ci un sistem integrat în care modificările se propagă între module.

Din punct de vedere academic, lucrarea arată cum se poate construi o platformă web care combină concepte de management agile, arhitectură full-stack și funcții asistate de AI. Din punct de vedere practic, SDLC Hub oferă o bază extensibilă pentru echipe care vor să controleze procesul de lucru fără să piardă flexibilitatea

metodologică.

4.2 Direcții viitoare

Aplicația poate fi extinsă prin automatizarea mai avansată a testelor end-to-end, astfel încât fluxurile principale să fie verificate constant după fiecare modificare. O altă direcție este integrarea cu calendare externe, precum Google Calendar sau Outlook, pentru ca evenimentele din proiect să poată fi sincronizate cu programul real al membrilor echipei.

Pe partea DevOps, aplicația poate fi extinsă cu integrare GitLab sau Bitbucket, nu doar GitHub. De asemenea, workflow-urile pot deveni mai configurabile, cu reguli mai detaliate pentru tranziții, limite WIP și automatizări. O direcție importantă este și dezvoltarea unor analize mai avansate pentru predictibilitate, risc și capacitate, astfel încât sistemul să poată ajuta echipa nu doar să vadă starea proiectului, ci și să anticipeze problemele înainte ca acestea să afecteze livrarea.

O direcție utilă este dezvoltarea unui sistem de template-uri pentru proiecte. În acest mod, o echipă ar putea porni rapid de la o configurație existentă, cu roluri, workflow-uri, permisiuni și rapoarte predefinite. Template-urile ar reduce timpul de configurare și ar permite reutilizarea unor practici bune între proiecte similare.

Funcțiile AI pot fi extinse printr-un mecanism de feedback. Utilizatorul ar putea marca o recomandare ca utilă sau inutilă, iar sistemul ar putea folosi aceste informații pentru a ajusta prompturile și regulile locale. De asemenea, explicațiile AI ar putea include surse interne din proiect, precum task-uri similare, estimări anterioare sau riscuri întâlnite în sprinturi precedente.

Într-o versiune viitoare, platforma poate include și suport mai avansat pentru organizații. Acesta ar putea adăuga spații de lucru, politici comune, raportare cross-proiect și management de portofoliu. O astfel de extindere ar păstra principiile aplicației, dar ar permite coordonarea mai multor proiecte în același ecosistem.

Bibliografie

- [1] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum. <https://scrumguides.org/scrum-guide.html>, 2020.
- [2] Daniel Vacanti and Yuval Yeret. The kanban guide. <https://kanbanguides.org/>, 2020.
- [3] Mariyam Mahboob, Mohammed Rayyan Uddin Ahmed, Zoiba Zia, Mariam Shakeel Ali, and Ayman Khaleel Ahmed. Future of artificial intelligence in agile software development. <https://arxiv.org/abs/2408.00703>, 2024.
- [4] Atlassian. Jira software features. <https://www.atlassian.com/software/jira/features>, 2026.
- [5] Atlassian. Trello guide. <https://trello.com/guide>, 2026.
- [6] GitHub Docs. About projects. <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2026.
- [7] Howard Lei, Farnaz Ganjeizadeh, Pradeep Kumar Jayachandran, and Pinar Ozcan. A statistical analysis of the effects of scrum and kanban on software development projects. *Robotics and Computer-Integrated Manufacturing*, 43:59–67, 2017.
- [8] IBM. What is three-tier architecture? <https://www.ibm.com/topics/three-tier-architecture>, 2026.
- [9] Vercel. Next.js documentation. <https://nextjs.org/docs>, 2026.
- [10] Meta Open Source. React documentation. <https://react.dev/>, 2026.
- [11] Tailwind Labs. Tailwind css documentation. <https://tailwindcss.com/docs>, 2026.
- [12] dnd kit. dnd kit documentation. <https://docs.dndkit.com/>, 2026.
- [13] Recharts. Recharts documentation. <https://recharts.org/en-US/>, 2026.

- [14] Tiptap. Tiptap documentation. <https://tiptap.dev/docs>, 2026.
- [15] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026.
- [16] SQLAlchemy. Ssqlalchemy documentation. <https://docs.sqlalchemy.org/>, 2026.
- [17] SQLAlchemy. Alembic documentation. <https://alembic.sqlalchemy.org/en/latest/>, 2026.
- [18] Michael B. Jones, John Bradley, and Nat Sakimura. RFC 7519: Json web token (jwt). <https://www.rfc-editor.org/rfc/rfc7519>, 2015.
- [19] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026.
- [20] TanStack. Tanstack query documentation. <https://tanstack.com/query/latest/docs/framework/react/overview>, 2026.
- [21] GitHub Docs. Webhooks documentation. <https://docs.github.com/en/webhooks>, 2026.
- [22] MDN Web Docs. The websocket api. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API, 2026.